

Assignment /

Assignment #: 23

Topic : Exception Handling

Level 1: Easy

1. Write a Python function `safe_division(a, b)` that divides `a` by `b`. Handle the `ZeroDivisionError` exception by printing "Error: Division by zero!" if `b` is zero.

Sample test Cases:

Test Cases	Test Case1:	Test Case 2:
Input	10, 2	8, 0
Output	5.0	Error: Division by zero!

2. Write a Python function `get_positive_integer()` that prompts the user to enter a positive integer. Handle the `ValueError` exception by printing "Error: Invalid input! Please enter a positive integer."

Sample test Cases:

Test Cases	Test Case1:	Test Case 2:
Input	5	abc
Output	5	Error: Invalid input! Please enter a positive integer.

3. Write a Python program to access the 5th element of a list. Handle the `IndexError` exception by printing "Error: Index out of range!" if the index is not valid.

Sample test Cases:

Test Cases	Test Case1:	Test Case 2:
Input	[1, 2, 3, 4, 5, 6]	[1, 2, 3, 4]
Output	5	Error: Index out of range!

4. Write a Python function `multiply(a, b)` that multiplies two numbers `a` and `b`. Handle the `TypeError` exception by printing "Error: Invalid operand type!" if either `a` or `b` is not a number.

Sample test Cases:

Test Cases	Test Case1:	Test Case 2:
Input	3, 4	"3", 4
Output	12	Error: Invalid operand type!

5. Write a Python program that tries to access a key "name" from a dictionary `person`. Handle the `KeyError` exception by printing "Error: Key not found!" if the key does not exist.

Sample test Cases:

Test Cases	Test Case1:	Test Case 2:
Input	Access key "name" from {"name": "Alice", "age": 30}	Access key "name" from {"age": 30}
Output	Alice	Error: Key not found!

Level 2: Medium

1. Write a Python program to input citizen details including Aadhar number, name, city, state, and country. Define a custom exception `NoMatchException` that should

be raised if the country name is not equal to "India". Handle this exception and print the citizen details only if the country is "India".

Sample test Cases:

Test Cases	Test Case1:	Test Case 2:
Input	Aadhar Number: 123456789012 Name: John Doe City: Bangalore State: Karnataka Country: India	Aadhar Number: 987654321098 Name: Jane Smith City: New York State: NY Country: USA
Output	Citizen Details: Aadhar Number: 123456789012 Name: John Doe City: Bangalore State: Karnataka Country: India	Country name does not match 'India'

2. Write a Python program to create a calculator application that performs basic operations like addition, subtraction, multiplication, and division with the following criteria:

- **Input must be in number format. No character or string inputs are accepted.**
- **For division, the denominator should not be zero.**

- For multiplication, both the multiplier and multiplicand should not be 0 or 1.

Define and handle user-defined exceptions (InputNotANumberException, DivisionByZeroException, InvalidMultiplierException) wherever necessary.

Sample test Cases:

Test Cases	Test Case1:	Test Case 2:
Input	Operation: Addition (+) Operand 1: 10 Operand 2: 5	Operation: Division (/) Operand 1: 10 Operand 2: 0
Output	Input: 10 + 5 Output: 15	Error: Division by zero is not allowed.

3. You are developing a Java application for a bookstore that manages book inventory. As part of the application, you need to handle various exceptions that may occur during the inventory management process. Design and implement exception handling for the following scenarios:

1. **InputMismatchException Handling:** When the user inputs data for book quantity, there is a possibility of encountering an InputMismatchException if the input provided is not a valid integer. Implement exception handling to catch and handle this exception gracefully. Display an error message informing the user about the incorrect input format and prompt them to enter the quantity again.
2. **NumberFormatException Handling:** When processing book prices, there is a risk of encountering a NumberFormatException if the price entered by the

user cannot be parsed as a valid decimal number. Implement exception handling to catch and handle this exception. Display an error message indicating that the price format is invalid and prompt the user to enter the price again.

3. **ArrayIndexOutOfBoundsException Handling:** During the inventory update process, there is a possibility of encountering an **ArrayIndexOutOfBoundsException** if the user attempts to access an array element with an invalid index. Implement exception handling to catch and handle this exception. Display an error message indicating that the specified index is out of bounds and prompt the user to enter a valid index

Sample test Cases:

Test Cases	Test Case1:	Test Case 2:
Input	User enters book quantity as "abc" instead of an integer.	User enters book price as "20.0.0" which is not a valid decimal number.
Output	Enter quantity of books: abc Error: Input should be a valid integer. Please enter quantity again.	Enter price of the book: 20.0.0 Error: Invalid price format. Please enter price again.

4. **Write a Python program to determine whether a given integer is a palindrome.**

A palindrome is a number that remains the same when its digits are reversed. For example, 121 and 1331 are palindromes, but 123 and 456 are not.

Constraints

$1 \leq N \leq 100$

Sample test Cases

Test Cases	Test Case 1:	Test Case 2:	Test Case 3:	Test Case 4:
N	153	666	hello	1.022
Output	Not Palindrome	Palindrome	Enter only integer numbers	Enter only integer numbers

5. Write a Python program to calculate the sum of the series: $1 + 1/2^2 + 1/3^3 + \dots + 1/n^n$. The program should also ensure that a positive integer is entered as input. If a non-positive value is entered, an exception should be raised with the message "Enter only a positive value."

Constraints

$1 \leq N \leq 100$

Sample test Cases

Test Cases	Test Case 1:	Test Case 2:	Test Case 3:	Test Case 4:
N	-2	5	-1	6
Output	Exception: Enter only positive numbers	1.29126	Exception: Enter only positive numbers	1.29128

Level 3: Hard

1. Check whether Shalini is eligible for professional course based on the following criteria:

Marks in Maths ≥ 65

Marks in Phy $\geq$ 55

Marks in Chem $\geq$ 50

Total in all three subject $\geq$ 180

Input format

The first line of each test case consists of maths mark.

The second line of each test case consists of physics mark.

The third line of each test case consists of chemistry mark.

Output format

Print Eligible if

Marks in Maths $\geq$ 65

Marks in Phy $\geq$ 55

Marks in Chem $\geq$ 50

Total in all three subject $\geq$ 180 else raise an exception not eligible.

Constraints

1 $\leq$ M $\leq$ 100

1 $\leq$ P $\leq$ 100

1 $\leq$ C $\leq$ 100

Sample test Cases

Test Cases	Test Case1:	Test Case 2:	Test Case 3:	Test Case 4:
M	65	60	65	95
P	57	25	55	97
C	62	66	50	92
Output	Eligible	Not Eligible	Not Eligible	Eligible

2. Create a program that asks the user for a number and tries to convert it to an integer. If the conversion is successful, print the square of the number. If not, display an error message.

Also, ensure the program always prints "Execution complete" whether an exception occurs or not.

Input format

The user should enter a valid integer. If the input is not a valid integer, the program will display an error message.

Output format

If the input is a valid integer, it will output the square of the number.

If the input is not a valid integer (e.g., contains non-numeric characters), it will output an error message.

In both cases, the program will conclude by printing "Execution complete."

Constraints

The input must be a whole number (integer). ($1 \leq N \leq 100$)

The program will handle any valid integer within the range of Python's integer data type.

Sample test Cases

Test Cases	Test Case 1:	Test Case 2:	Test Case 4:
Input (N)	Enter a number: 5	Enter a number: abc	Enter a number: 3.14
Output	The square of 5 is 25 Execution complete	Error: Invalid input. Please enter a valid number. Execution complete	Error: Invalid input. Please enter a valid number. Execution complete

3. Write a Python program for sum of squares of first n natural numbers

Input format

The first line of each test case consists of integer N.

Output format

Print the sum of the squares for N natural numbers else throw an exception as enter the value within N.

Constraints

$1 \leq N \leq 100$

Sample test Cases

Test Cases	Test Case 1:	Test Case 2:	Test Case 3:	Test Case 4:
N	5	7	102	10
Output	1 5 14 30 55	1 5 14 30 55 91 140	Enter value between 1 to 100	1 5 14 30 55 91 140 204 285 385

4. You are required to develop a Python program that simulates bank account management, focusing on transaction limits and balance checks. Implement custom exception handling to manage scenarios where transaction amounts exceed predefined limits or where there is insufficient balance in the account.

Requirements:

Custom Exception Handling: Implement a custom exception class `PayOutOfBoundsException` that extends Python's built-in `Exception`. This exception should be used to handle situations where:

- The transaction amount exceeds the maximum limit (30,000).
- The account balance is insufficient to cover the transaction amount.

Account Management Class: Create a class named **AccountManagement** to manage bank account transactions with the following attributes:

- **current_balance:** Initialized to 80,000, representing the current account balance.
- **max_transaction_limit:** Set to 30,000, indicating the maximum amount that can be withdrawn in a single transaction.

Sample test Cases

Test Cases	Test Case 1:	Test Case 2:
N	Withdraw amount = 20000	Withdraw amount = 35000
Output	Withdrawal successful. Updated balance: 60000	Error: Transaction amount exceeds insufficient balance.

5. You are tasked with developing a Python program to validate user authentication based on provided usernames and passwords. Implement custom exception handling to manage scenarios where usernames and passwords do not meet specified criteria.

Requirements:

1. Custom Exceptions:

Implement two custom exception classes:

- **InvalidUsernameException:** Raised when the username does not meet specified constraints.
- **InvalidPasswordException:** Raised when the password does not meet specified constraints.

Username Validation Criteria:

- The username must consist of 6 to 30 characters inclusive.
- It can only contain alphanumeric characters (a-z, A-Z, 0-9) and underscores (_).

- The first character must be an alphabetic character (a-z or A-Z).

Password Validation Criteria:

- The password must be at least 8 characters long.
- It must contain at least one lowercase English character (a-z).
- It must contain at least one uppercase English character (A-Z).
- It must contain at least one digit (0-9).
- It must contain at least one special character from the set: !@#\$%^&*()-+.

Functionality:

- Create functions `validate_username(username)` and `validate_password(password)` to check the respective criteria.
- Implement a function `validate_login(username, password)` that uses the above functions to validate both username and password:
- If valid, print a message "Welcome 'username'".
- If invalid, catch and print the appropriate exception message (`InvalidUsernameException` or `InvalidPasswordException`).

Sample test Cases

Test Cases	Test Case 1:	Test Case 2:
N	Username: "User123" Password: "Password@1"	Username: "User" Password: "Password@1"
Output	Welcome 'User123'	Invalid username or password. (Reason: InvalidUsernameException: Username length should be between 6 and 30 characters.)